

CHAPTER

7

SINGLE-DIMENSIONAL ARRAYS

Objectives

- To describe why arrays are necessary in programming (§7.1).
- To declare array reference variables and create arrays (§§7.2.1 and 7.2.2).
- To obtain array size using `arrayRefVar.length` and know default values in an array (§7.2.3).
- To access array elements using indexes (§7.2.4).
- To declare, create, and initialize an array using an array initializer (§7.2.5).
- To program common array operations (displaying arrays, summing all elements, finding the minimum and maximum elements, random shuffling, and shifting elements) (§7.2.6).
- To simplify programming using the foreach loops (§7.2.7).
- To apply arrays in application development (`AnalyzeNumbers`, and `DeckOfCards`) (§§7.3 and 7.4).
- To copy contents from one array to another (§7.5).
- To develop and invoke methods with array arguments and return values (§§7.6–7.8).
- To define a method with a variable-length argument list (§7.9).
- To search elements using the linear (§7.10.1) or binary (§7.10.2) search algorithm.
- To sort an array using the selection sort approach (§7.11).
- To use the methods in the `java.util.Arrays` class (§7.12).
- To pass arguments to the main method from the command line (§7.13).



- c. `public static void main(String x[])`
- d. `static void main(String x[])`

7.13.2 Show the output of the following program when invoked using

1. `java Test I have a dream`
2. `java Test "1 2 3"`
3. `java Test`

```
public class Test {
    public static void main(String[] args) {
        System.out.println("Number of strings is " + args.length);
        for (int i = 0; i < args.length; i++)
            System.out.println(args[i]);
    }
}
```

KEY TERMS

anonymous array 262	indexed variable 252
array 250	linear search 269
array initializer 252	off-by-one error 255
binary search 269	postcondition 272
garbage collection 260	precondition 272
index 250	selection sort 273

CHAPTER SUMMARY

1. A variable is declared as an *array* type using the syntax `elementType[] arrayRefVar` or `elementType arrayRefVar[]`. The style `elementType[] arrayRefVar` is preferred, although `elementType arrayRefVar[]` is legal.
2. Unlike declarations for primitive data type variables, the declaration of an array variable does not allocate any space in memory for the array. An array variable is not a primitive data type variable. An array variable contains a reference to an array.
3. You cannot assign elements to an array unless it has already been created. You can create an array by using the `new` operator with the following syntax: `new elementType[arraySize]`.
4. Each element in the array is represented using the syntax `arrayRefVar[index]`. An *index* must be an integer or an integer expression.
5. After an array is created, its size becomes permanent and can be obtained using `arrayRefVar.length`. Since the index of an array always begins with `0`, the last index is always `arrayRefVar.length - 1`. An out-of-bounds error will occur if you attempt to reference elements beyond the bounds of an array.
6. Programmers often mistakenly reference the first element in an array with index `1`, but it should be `0`. This is called the *index off-by-one error*.

7. When an array is created, its elements are assigned the default value of **0** for the numeric primitive data types, **\u0000** for char types, and **false** for **boolean** types.
8. Java has a shorthand notation, known as the *array initializer*, which combines declaring an array, creating an array, and initializing an array in one statement, using the syntax `elementType[] arrayRefVar = {value0, value1, . . . , valuek}`.
9. When you pass an array argument to a method, you are actually passing the reference of the array; that is, the called method can modify the elements in the caller's original array.
10. If an array is sorted, *binary search* is more efficient than *linear search* for finding an element in the array.
11. *Selection sort* finds the smallest number in the list and swaps it with the first element. It then finds the smallest number remaining and swaps it with the first element in the remaining list, and so on, until only a single number remains.



QUIZ

Answer the quiz for this chapter online at the Companion Website.

PROGRAMMING EXERCISES

Sections 7.2–7.5

- *7.1** (*Assign grades*) Write a program that reads student scores, gets the best score, and then assigns grades based on the following scheme:

Grade is A if score is $\geq$ best -10 ;

Grade is B if score is $\geq$ best -20 ;

Grade is C if score is $\geq$ best -30 ;

Grade is D if score is $\geq$ best -40 ;

Grade is F otherwise.

The program prompts the user to enter the total number of students, then prompts the user to enter all of the scores, and concludes by displaying the grades. Here is a sample run:



```
Enter the number of students: 4 [Enter]
Enter 4 scores: 40 55 70 58 [Enter]
Student 0 score is 40 and grade is C
Student 1 score is 55 and grade is B
Student 2 score is 70 and grade is A
Student 3 score is 58 and grade is B
```

- 7.2** (*Reverse the numbers entered*) Write a program that reads 10 integers then displays them in the reverse of the order in which they were read.

CHAPTER

8

MULTIDIMENSIONAL ARRAYS

Objectives

- To give examples of representing data using two-dimensional arrays (§8.1).
- To declare variables for two-dimensional arrays, create arrays, and access array elements in a two-dimensional array using row and column indices (§8.2).
- To program common operations for two-dimensional arrays (displaying arrays, summing all elements, finding the minimum and maximum elements, and random shuffling) (§8.3).
- To pass two-dimensional arrays to methods (§8.4).
- To write a program for grading multiple-choice questions using two-dimensional arrays (§8.5).
- To solve the closest pair problem using two-dimensional arrays (§8.6).
- To check a Sudoku solution using two-dimensional arrays (§8.7).
- To use multidimensional arrays (§8.8).



```

46     }
47
48     System.out.println("Your birthday is " + day);
49 }
50 }

```

A three-dimensional array **dates** is created in lines 8–28. This array stores five sets of numbers. Each set is a 4-by-4 two-dimensional array.

The loop starting from line 33 displays the numbers in each set and prompts the user to answer whether the birthday is in the set (lines 41 and 42). If the day is in the set, the first number (**dates[i][0][0]**) in the set is added to variable **day** (line 45).

8.8.1 Declare an array variable for a three-dimensional array, create a $4 \times 6 \times 5$ **int** array, and assign its reference to the variable.

8.8.2 Assume **char[][][] x = new char[12][5][2]**, how many elements are in the array? What are **x.length**, **x[2].length**, and **x[0][0].length**?

8.8.3 Show the output of the following code:

```

int[][][] array = {{{1, 2}, {3, 4}}, {{5, 6},{7, 8}}};
System.out.println(array[0][0][0]);
System.out.println(array[1][1][1]);

```



KEY TERMS

column index 290

multidimensional array 303

nested array 290

row index 291

two-dimensional array 290

CHAPTER SUMMARY

1. A two-dimensional array can be used to store a table.
2. A variable for two-dimensional arrays can be declared using the syntax: **elementType[][] arrayVar**.
3. A two-dimensional array can be created using the syntax: **new elementType[ROW_SIZE][COLUMN_SIZE]**.
4. Each element in a two-dimensional array is represented using the syntax: **arrayVar[rowIndex][columnIndex]**.
5. You can create and initialize a two-dimensional array using an array initializer with the syntax: **elementType[][] arrayVar = {{row values}, . . . , {row values}}**.
6. You can use arrays of arrays to form multidimensional arrays. For example, a variable for three-dimensional arrays can be declared as **elementType[][][] arrayVar** and a three-dimensional array can be created using **new elementType[size1][size2][size3]**.

QUIZ

Answer the quiz for this chapter online at the book Companion Website.



CHAPTER 9

OBJECTS AND CLASSES

Objectives

- To describe objects and classes, and use classes to model objects (§9.2).
- To use UML graphical notation to describe classes and objects (§9.2).
- To demonstrate how to define classes and create objects (§9.3).
- To create objects using constructors (§9.4).
- To define a reference variable using a reference type and access objects via object reference variables (§9.5).
- To access an object's data and methods using the object member access operator (.) (§9.5.1).
- To define data fields of reference types and assign default values for an object's data fields (§9.5.2).
- To distinguish between object reference variables and primitive-data-type variables (§9.5.3).
- To use the Java library classes **Date**, **Random**, and **Point2D** (§9.6).
- To distinguish between instance and static variables and methods (§9.7).
- To define private data fields with appropriate getter and setter methods (§9.8).
- To encapsulate data fields to make classes easy to maintain (§9.9).
- To develop methods with object arguments and differentiate between primitive-type arguments and object-type arguments (§9.10).
- To store and process objects in arrays (§9.11).
- To create immutable objects from immutable classes to protect the contents of objects (§9.12).
- To determine the scope of variables in the context of a class (§9.13).
- To use the keyword **this** to refer to the calling object itself (§9.14).



```

    public void m2() {
        Test.id = 45;
    }
}

```

Key Terms

accessor 347	instantiation 324
action 324	mutator 347
anonymous object 333	no-arg constructor 327
attribute 324	null value 334
behavior 324	object 324
class 324	object-oriented programming (OOP) 324
class's variable 357	package-private (or package-access) 344
client 327	private constructor 346
constructor 324	property 324
data field 324	public class 327
data field encapsulation 346	reference type 332
default constructor 331	reference variable 332
dot operator (.) 333	setter 347
getter 347	state 324
immutable class 355	static method 339
immutable object 355	static variable 339
instance 324	this keyword 358
instance method 333	Unified Modeling Language (UML) 325
instance variable 333	

Chapter Summary

-
1. A *class* is a template for *objects*. It defines the *properties* of objects and provides *constructors* for creating objects and methods for manipulating them.
 2. A class is also a data type. You can use it to declare object *reference variables*. An object reference variable that appears to hold an object actually contains a reference to that object. Strictly speaking, an object reference variable and an object are different, but most of the time the distinction can be ignored.
 3. An object is an *instance* of a class. You use the **new** operator to create an object and the *dot operator* (.) to access members of that object through its reference variable.
 4. An *instance variable* or *method* belongs to an instance of a class. Its use is associated with individual instances. A *static variable* is a variable shared by all instances of the same class. A *static method* is a method that can be invoked without using instances.
 5. Every instance of a class can access the class's static variables and methods. For clarity, however, it is better to invoke static variables and methods using **ClassName.variable** and **ClassName.method**.
 6. Visibility modifiers specify how the class, method, and data are accessed. A **public** class, method, or data is accessible to all clients. A **private** method or data is accessible only inside the class.
 7. You can provide a getter (accessor) method or a setter (mutator) method to enable clients to see or modify the data.

8. A getter method has the signature `public returnType getPropertyName()`. If the `returnType` is `boolean`, the getter method should be defined as `public boolean isPropertyName()`. A setter method has the signature `public void setPropertyName(dataType propertyValue)`.
9. All parameters are passed to methods using pass-by-value. For a parameter of a primitive type, the actual value is passed; for a parameter of a *reference type*, the reference for the object is passed.
10. A Java array is an object that can contain primitive-type values or object-type values. When an array of objects is created, its elements are assigned the default value of `null`.
11. Once it is created, an *immutable object* cannot be modified. To prevent users from modifying an object, you can define *immutable classes*.
12. The scope of instance and static variables is the entire class, regardless of where the variables are declared. Instance and static variables can be declared anywhere in the class. For consistency, they are declared at the beginning of the class in this book.
13. The keyword `this` can be used to refer to the calling object. It can also be used inside a constructor to invoke another constructor of the same class.



Quiz

Answer the quiz for this chapter online at the book Companion Website.

PROGRAMMING EXERCISES

three objectives



Pedagogical Note

The exercises in Chapters 9–13 help you to achieve three objectives:

1. Design classes and draw UML class diagrams.
2. Implement classes from the UML.
3. Use classes to develop applications.

Students can download solutions for the UML diagrams for the even-numbered exercises from the Companion Website and instructors can download all solutions from the Instructor Website.

Starting from Section 9.7, all data fields should be declared private and all constructors and methods should be defined public unless specified otherwise.

Sections 9.2–9.5

9.1 (The *Rectangle* class) Following the example of the *Circle* class in Section 9.2, design a class named *Rectangle* to represent a rectangle. The class contains:

- Two `double` data fields named `width` and `height` that specify the width and height of the rectangle. The default values are `1` for both `width` and `height`.
- A no-arg constructor that creates a default rectangle.
- A constructor that creates a rectangle with the specified `width` and `height`.
- A method named `getArea()` that returns the area of this rectangle.
- A method named `getPerimeter()` that returns the perimeter.

Draw the UML diagram for the class then implement the class. Write a test program that creates two *Rectangle* objects—one with width `4` and height `40`, and

CHAPTER 10

OBJECT-ORIENTED THINKING

Objectives

- To apply class abstraction to develop software (§10.2).
- To explore the differences between the procedural paradigm and object-oriented paradigm (§10.3).
- To discover the relationships between classes (§10.4).
- To design programs using the object-oriented paradigm (§§10.5 and 10.6).
- To create objects for primitive values using the wrapper classes (**Byte**, **Short**, **Integer**, **Long**, **Float**, **Double**, **Character**, and **Boolean**) (§10.7).
- To simplify programming using automatic conversion between primitive types and wrapper class types (§10.8).
- To use the **BigInteger** and **BigDecimal** classes for computing very large numbers with arbitrary precisions (§10.9).
- To use the **String** class to process immutable strings (§10.10).
- To use the **StringBuilder** and **StringBuffer** classes to process mutable strings (§10.11).



KEY TERMS

abstract data type (ADT) 368	composition 376
aggregation 376	has-a relationship 376
boxing 386	multiplicity 375
class abstraction 368	stack 380
class encapsulation 368	unboxing 386
class's contract 368	

CHAPTER SUMMARY

-
1. The procedural paradigm focuses on designing methods. The object-oriented paradigm couples data and methods together into objects. Software design using the object-oriented paradigm focuses on objects and operations on objects. The object-oriented approach combines the power of the procedural paradigm with an added dimension that integrates data with operations into objects.
 2. Many Java methods require the use of objects as arguments. Java offers a convenient way to incorporate, or wrap, a primitive data type into an object (e.g., wrapping `int` into the `Integer` class, and wrapping `double` into the `Double` class).
 3. Java can automatically convert a primitive-type value to its corresponding wrapper object in the context and vice versa.
 4. The `BigInteger` class is useful for computing and processing integers of any size. The `BigDecimal` class can be used to compute and process floating-point numbers with any arbitrary precision.
 5. A `String` object is immutable; its contents cannot be changed. To improve efficiency and save memory, the JVM stores two literal strings that have the same character sequence in a unique object. This unique object is called an *interned string object*.
 6. A *regular expression* (abbreviated *regex*) is a string that describes a pattern for matching a set of strings. You can match, replace, or split a string by specifying a pattern.
 7. The `StringBuilder` and `StringBuffer` classes can be used to replace the `String` class. The `String` object is immutable, but you can add, insert, or append new contents into `StringBuilder` and `StringBuffer` objects. Use `String` if the string contents do not require any change and use `StringBuilder` or `StringBuffer` if they might change.

QUIZ

Answer the quiz for this chapter online at the book Companion Website.



PROGRAMMING EXERCISES

MyProgrammingLab™

Sections 10.2 and 10.3

***10.1** (The `Time` class) Design a class named `Time`. The class contains:

- The data fields `hour`, `minute`, and `second` that represent a time.
- A no-arg constructor that creates a `Time` object for the current time. (The values of the data fields will represent the current time.)

CHAPTER 11

INHERITANCE AND POLYMORPHISM

Objectives

- To define a subclass from a superclass through inheritance (§11.2).
- To invoke the superclass's constructors and methods using the **super** keyword (§11.3).
- To override instance methods in the subclass (§11.4).
- To distinguish differences between overriding and overloading (§11.5).
- To explore the **toString()** method in the **Object** class (§11.6).
- To discover polymorphism and dynamic binding (§§11.7 and 11.8).
- To describe casting and explain why explicit downcasting is necessary (§11.9).
- To explore the **equals** method in the **Object** class (§11.10).
- To store, retrieve, and manipulate objects in an **ArrayList** (§11.11).
- To construct an array list from an array, to sort and shuffle a list, and to obtain max and min element from a list (§11.12).
- To implement a **Stack** class using **ArrayList** (§11.13).
- To enable data and methods in a superclass accessible from subclasses using the **protected** visibility modifier (§11.14).
- To prevent class extending and method overriding using the **final** modifier (§11.15).



CHAPTER SUMMARY

1. You can define a new class from an existing class. This is known as class *inheritance*. The new class is called a *subclass*, *child class*, or *extended class*. The existing class is called a *superclass*, *parent class*, or *base class*.
2. A constructor is used to construct an instance of a class. Unlike properties and methods, the constructors of a superclass are not inherited in the subclass. They can be invoked only from the constructors of the subclasses, using the keyword **super**.
3. A constructor may invoke an overloaded constructor or its superclass's constructor. The call must be the first statement in the constructor. If none of them is invoked explicitly, the compiler puts **super()** as the first statement in the constructor, which invokes the superclass's no-arg constructor.
4. To *override* a method, the method must be defined in the subclass using the same signature and the same or compatible return type as in its superclass.
5. An instance method can be overridden only if it is accessible. Thus, a private method cannot be overridden because it is not accessible outside its own class. If a method defined in a subclass is private in its superclass, the two methods are completely unrelated.
6. Like an instance method, a static method can be inherited. However, a static method cannot be overridden. If a static method defined in the superclass is redefined in a subclass, the method defined in the superclass is hidden.
7. Every class in Java is descended from the **java.lang.Object** class. If no superclass is specified when a class is defined, its superclass is **Object**.
8. If a method's parameter type is a superclass (e.g., **Object**), you may pass an object to this method of any of the parameter's subclasses (e.g., **Circle** or **String**). This is known as polymorphism.
9. It is always possible to cast an instance of a subclass to a variable of a superclass because an instance of a subclass is *always* an instance of its superclass. When casting an instance of a superclass to a variable of its subclass, explicit casting must be used to confirm your intention to the compiler with the (**SubclassName**) cast notation.
10. A class defines a type. A type defined by a subclass is called a *subtype*, and a type defined by its superclass is called a *supertype*.
11. When invoking an instance method from a reference variable, the *actual type* of the variable decides which implementation of the method is used *at runtime*. This is known as dynamic binding.
12. You can use **obj instanceof AClass** to test whether an object is an instance of a class.
13. You can use the **ArrayList** class to create an object to store a list of objects.
14. You can use the **protected** modifier to prevent the data and methods from being accessed by nonsubclasses from a different package.
15. You can use the **final** modifier to indicate a class is final and cannot be extended and to indicate a method is final and cannot be overridden.

CHAPTER 13

ABSTRACT CLASSES AND INTERFACES

Objectives

- To design and use abstract classes (§13.2).
- To generalize numeric wrapper classes **BigInteger** and **BigDecimal** using the abstract **Number** class (§13.3).
- To process a calendar using the **Calendar** and **GregorianCalendar** classes (§13.4).
- To specify common behavior for objects using interfaces (§13.5).
- To define interfaces and define classes that implement interfaces (§13.5).
- To define a natural order using the **Comparable** interface (§13.6).
- To make objects cloneable using the **Cloneable** interface (§13.7).
- To explore the similarities and differences among concrete classes, abstract classes, and interfaces (§13.8).
- To design the **Rational** class for processing rational numbers (§13.9).
- To design classes that follow the class-design guidelines (§13.10).



13.10.8 Interfaces vs. Abstract Classes

Both interfaces and abstract classes can be used to specify common behavior for objects. How do you decide whether to use an interface or a class? In general, a strong is-a relationship that clearly describes a parent-child relationship should be modeled using classes. For example, since an orange is a fruit, their relationship should be modeled using class inheritance. A weak is-a relationship, also known as an is-kind-of relationship, indicates that an object possesses a certain property. A weak is-a relationship can be modeled using interfaces. For example, all strings are comparable, so the `String` class implements the `Comparable` interface. A circle or a rectangle is a geometric object, so `Circle` can be designed as a subclass of `GeometricObject`. Circles are different and comparable based on their radii, so `Circle` can implement the `Comparable` interface.

Interfaces are more flexible than abstract classes because a subclass can extend only one superclass, but can implement any number of interfaces. However, interfaces cannot contain data fields. In Java 8, interfaces can contain default methods and static methods, which are very useful to simplify class design. We will give examples of this type of design in Chapter 20, Lists, Stacks, Queues, and Priority Queues.

13.10.1 Describe class-design guidelines.



KEY TERMS

abstract class	500	marker interface	518
abstract method	500	shallow copy	520
deep copy	520	subinterface	523
interface	523		

CHAPTER SUMMARY

1. *Abstract classes* are like regular classes with data and methods, but you cannot create instances of abstract classes using the `new` operator.
2. An *abstract method* cannot be contained in a nonabstract class. If a subclass of an abstract superclass does not implement all the inherited abstract methods of the superclass, the subclass must be defined as abstract.
3. A class that contains abstract methods must be abstract. However, it is possible to define an abstract class that doesn't contain any abstract methods.
4. A subclass can be abstract even if its superclass is concrete.
5. An *interface* is a class-like construct that contains only constants, abstract methods, default methods, and static methods. In many ways, an interface is similar to an abstract class, but an abstract class can contain data fields.
6. An interface is treated like a special class in Java. Each interface is compiled into a separate bytecode file, just like a regular class.
7. The `java.lang.Comparable` interface defines the `compareTo` method. Many classes in the Java library implement `Comparable`.

8. The `java.lang.Cloneable` interface is a *marker interface*. An object of the class that implements the `Cloneable` interface is cloneable.
9. A class can extend only one superclass but can implement one or more interfaces.
10. An interface can extend one or more interfaces.

Quiz

Answer the quiz for this chapter online at the book Companion Website.



PROGRAMMING EXERCISES

MyProgrammingLab™

Sections 13.2 and 13.3

****13.1** (*Triangle class*) Design a new `Triangle` class that extends the abstract `GeometricObject` class. Draw the UML diagram for the classes `Triangle` and `GeometricObject` then implement the `Triangle` class. Write a test program that prompts the user to enter three sides of the triangle, a color, and a Boolean value to indicate whether the triangle is filled. The program should create a `Triangle` object with these sides, and set the `color` and `filled` properties using the input. The program should display the area, perimeter, color, and true or false to indicate whether it is filled or not.

***13.2** (*Shuffle ArrayList*) Write the following method that shuffles an `ArrayList` of numbers:

```
public static void shuffle(ArrayList<Number> list)
```

***13.3** (*Sort ArrayList*) Write the following method that sorts an `ArrayList` of numbers:

```
public static void sort(ArrayList<Number> list)
```

****13.4** (*Display calendars*) Rewrite the `PrintCalendar` class in Listing 6.12 to display a calendar for a specified month using the `Calendar` and `GregorianCalendar` classes. Your program receives the month and year from the command line. For example:

```
java Exercise13_04 5 2016
```

This displays the calendar shown in Figure 13.9.

```

C:\> Command Prompt
c:\exercise>java Exercise13_04 10 2019
October 2019

-----
Sun Mon Tue Wed Thu Fri Sat
      1  2  3  4  5
 6  7  8  9 10 11 12
13 14 15 16 17 18 19
20 21 22 23 24 25 26
27 28 29 30 31

c:\exercise>
    
```

FIGURE 13.9 The program displays a calendar for May 2016.

CHAPTER 19

GENERIC

Objectives

- To describe the benefits of generics (§19.2).
- To use generic classes and interfaces (§19.2).
- To define generic classes and interfaces (§19.3).
- To explain why generic types can improve reliability and readability (§19.3).
- To define and use generic methods and bounded generic types (§19.4).
- To develop a generic sort method to sort an array of **Comparable** objects (§19.5).
- To use raw types for backward compatibility (§19.6).
- To explain why wildcard generic types are necessary (§19.7).
- To describe generic-type erasure and list certain restrictions and limitations on generic types caused by type erasure (§19.8).
- To design and implement generic matrix classes (§19.9).



```

20     GenericMatrix.printResult(
21         m1, m2, rationalMatrix.multiplyMatrix(m1, m2), '*');
22     }
23 }

```

multiply two matrices

```

m1 + m2 is
1/5 1/6 1/7      1/6 1/7 1/8      11/30 13/42 15/56
2/5 1/3 2/7  +  1/3 2/7 1/4  =  11/15 13/21 15/28
3/5 1/2 3/7      1/2 3/7 3/8      11/10 13/14 45/56

m1 * m2 is
1/5 1/6 1/7      1/6 1/7 1/8      101/630 101/735 101/840
2/5 1/3 2/7  *  1/3 2/7 1/4  =  101/315 202/735 101/420
3/5 1/2 3/7      1/2 3/7 3/8      101/210 101/245 101/280

```



- 19.9.1** Why are the **add**, **multiple**, and **zero** methods defined abstract in the **GenericMatrix** class?
- 19.9.2** How are the **add**, **multiple**, and **zero** methods implemented in the **IntegerMatrix** class?
- 19.9.3** How are the **add**, **multiple**, and **zero** methods implemented in the **RationalMatrix** class?
- 19.9.4** What would be wrong if the **printResult** method is defined as follows?

```

public static void printResult(
    E[][] m1, E[][] m2, E[][] m3, char op)

```



KEY TERMS

actual concrete type	752	lower bound wildcard	
bounded generic type	757	(<? super E>)	762
bounded wildcard		raw type	760
(<? extends E>)	762	unbounded wildcard (<?>)	762
formal generic type	752	type erasure	764
generic instantiation	752		

CHAPTER SUMMARY

1. *Generics* give you the capability to parameterize types. You can define a class or a method with generic types, which are substituted with concrete types.
2. The key benefit of generics is to enable errors to be detected at compile time rather than at runtime.
3. A generic class or method permits you to specify allowable types of objects that the class or method can work with. If you attempt to use a class or method with an incompatible object, the compiler will detect the error.
4. A generic type defined in a class, interface, or a static method is called a *formal generic type*, which can be replaced later with an *actual concrete type*. Replacing a generic type is called a *generic instantiation*.

5. A generic class such as `ArrayList` used without a type parameter is called a *raw type*. Use of raw types allows for backward compatibility with the earlier versions of Java.
6. A wildcard generic type has three forms: `?`, `? extends T`, and `? super T`, where `T` is a generic type. The first form, `?`, called an *unbounded wildcard*, is the same as `? extends Object`. The second form, `? extends T`, called a *bounded wildcard*, represents `T` or a subtype of `T`. The third form, `? super T`, called a *lower bound wildcard*, denotes `T` or a supertype of `T`.
7. Generics are implemented using an approach called *type erasure*. The compiler uses the generic-type information to compile the code but erases it afterward, so the generic information is not available at runtime. This approach enables the generic code to be backward compatible with the legacy code that uses raw types.
8. You cannot create an instance using a generic-type parameter such as `new E()`.
9. You cannot create an array using a generic-type parameter such as `new E[10]`.
10. You cannot use a generic-type parameter of a class in a static context.
11. Generic-type parameters cannot be used in exception classes.



QUIZ

Answer the quiz for this chapter online at the book Companion Website.

PROGRAMMING EXERCISES

- 19.1 (Revising Listing 19.1) Revise the `GenericStack` class in Listing 19.1 to implement it using an array rather than an `ArrayList`. You should check the array size before adding a new element to the stack. If the array is full, create a new array that doubles the current array size and copy the elements from the current array to the new array.
- 19.2 (Implement `GenericStack` using inheritance) In Listing 19.1, `GenericStack` is implemented using composition. Define a new stack class that extends `ArrayList`. Draw the UML diagram for the classes then implement `GenericStack`. Write a test program that prompts the user to enter five strings and displays them in reverse order.
- 19.3 (Distinct elements in `ArrayList`) Write the following method that returns a new `ArrayList`. The new list contains the nonduplicate elements from the original list.

```
public static <E> ArrayList<E> removeDuplicates(ArrayList<E> list)
```

- 19.4 (Generic linear search) Implement the following generic method for linear search:

```
public static <E extends Comparable<E>>
    int linearSearch(E[] list, E key)
```

CHAPTER 20

LISTS, STACKS, QUEUES, AND PRIORITY QUEUES

Objectives

- To explore the relationship between interfaces and classes in the Java Collections Framework hierarchy (§20.2).
- To use the common methods defined in the **Collection** interface for operating collections (§20.2).
- To use the **Iterator** interface to traverse the elements in a collection (§20.3).
- To use a foreach loop to traverse the elements in a collection (§20.3).
- To use a **forEach** method to perform an action on each element in a collection (§20.4).
- To explore how and when to use **ArrayList** or **LinkedList** to store a list of elements (§20.5).
- To compare elements using the **Comparable** interface and the **Comparator** interface (§20.6).
- To use the static utility methods in the **Collections** class for sorting, searching, shuffling lists, and finding the largest and smallest element in collections (§20.7).
- To develop a multiple bouncing balls application using **ArrayList** (§20.8).
- To distinguish between **Vector** and **ArrayList** and to use the **Stack** class for creating stacks (§20.9).
- To explore the relationships among **Collection**, **Queue**, **LinkedList**, and **PriorityQueue** and to create priority queues using the **PriorityQueue** class (§20.10).
- To use stacks to write a program to evaluate expressions (§20.11).



CHAPTER SUMMARY

1. The **Collection** interface defines the common operations for lists, vectors, stacks, queues, priority queues, and sets.
2. Each collection is **Iterable**. You can obtain its **Iterator** object to traverse all the elements in the collection.
3. All the concrete classes except **PriorityQueue** in the Java Collections Framework implement the **Cloneable** and **Serializable** interfaces. Thus, their instances can be cloned and serialized.
4. A list stores an ordered collection of elements. To allow duplicate elements to be stored in a collection, you need to use a list. A list not only can store duplicate elements but also allows the user to specify where they are stored. The user can access elements by an index.
5. Two types of lists are supported: **ArrayList** and **LinkedList**. **ArrayList** is a resizable-array implementation of the **List** interface. All the methods in **ArrayList** are defined in **List**. **LinkedList** is a *linked-list* implementation of the **List** interface. In addition to implementing the **List** interface, this class provides the methods for retrieving, inserting, and removing elements from both ends of the list.
6. **Comparator** can be used to compare the objects of a class that doesn't implement **Comparable**.
7. The **Vector** class extends the **AbstractList** class. Starting with Java 2, **Vector** has been the same as **ArrayList**, except that the methods for accessing and modifying the vector are synchronized. The **Stack** class extends the **Vector** class and provides several methods for manipulating the stack.
8. The **Queue** interface represents a queue. The **PriorityQueue** class implements **Queue** for a *priority queue*.



Quiz

Answer the quiz for this chapter online at the book Companion Website.

PROGRAMMING EXERCISES

Sections 20.2–20.7

- *20.1** (*Display words in ascending alphabetical order*) Write a program that reads words from a text file and displays all the words (duplicates allowed) in ascending alphabetical order. The words must start with a letter. The text file is passed as a command-line argument.

CHAPTER 21

SETS AND MAPS

Objectives

- To store unordered, nonduplicate elements using a set (§21.2).
- To explore how and when to use **HashSet** (§21.2.1), **LinkedHashSet** (§21.2.2), or **TreeSet** (§21.2.3) to store a set of elements.
- To compare the performance of sets and lists (§21.3).
- To use sets to develop a program that counts the keywords in a Java source file (§21.4).
- To tell the differences between **Collection** and **Map** and describe when and how to use **HashMap**, **LinkedHashMap**, or **TreeMap** to store values associated with keys (§21.5).
- To use maps to develop a program that counts the occurrence of the words in a text (§21.6).
- To obtain singleton sets, lists, and maps and unmodifiable sets, lists, and maps, use the static methods in the **Collections** class (§21.7).



KEY TERMS

hash map	831	read-only view	835
hash set	816	set	816
linked hash map	831	tree map	831
linked hash set	820	tree set	821
map	828		

CHAPTER SUMMARY

1. A set stores nonduplicate elements. To allow duplicate elements to be stored in a collection, you need to use a list.
2. A *map* stores key/value pairs. It provides a quick lookup for a value using a key.
3. Three types of sets are supported: **HashSet**, **LinkedHashSet**, and **TreeSet**. **HashSet** stores elements in an unpredictable order. **LinkedHashSet** stores elements in the order they were inserted. **TreeSet** stores elements sorted. **HashSet**, **LinkedHashSet**, and **TreeSet** are subtypes of **Collection**.
4. The **Map** interface maps keys to the elements. The keys are like indexes. In **List**, the indexes are integers. In **Map**, the keys can be any objects. A map cannot contain duplicate keys. Each key can map to at most one value. The **Map** interface provides the methods for querying, updating, and obtaining a collection of values and a set of keys.
5. Three types of maps are supported: **HashMap**, **LinkedHashMap**, and **TreeMap**. **HashMap** is efficient for locating a value, inserting an entry, and deleting an entry. **LinkedHashMap** supports ordering of the entries in the map. The entries in a **HashMap** are not ordered, but the entries in a **LinkedHashMap** can be retrieved either in the order in which they were inserted into the map (known as the *insertion order*) or in the order in which they were last accessed, from least recently accessed to most recently (*access order*). **TreeMap** is efficient for traversing the keys in a sorted order. The keys can be sorted using the **Comparable** interface or the **Comparator** interface.



QUIZ

Answer the quiz for this chapter online at the book Companion Website.

PROGRAMMING EXERCISES

Sections 21.2–21.4

- 21.1 (*Perform set operations on hash sets*) Create two linked hash sets {**"George"**, **"Jim"**, **"John"**, **"Blake"**, **"Kevin"**, **"Michael"**} and {**"George"**, **"Katie"**, **"Kevin"**, **"Michelle"**, **"Ryan"**} and find their union, difference, and intersection. (You can clone the sets to preserve the original sets from being changed by these set methods.)
- 21.2 (*Display nonduplicate words in ascending order*) Write a program that reads words from a text file and displays all the nonduplicate words in ascending order. The text file is passed as a command-line argument.
- **21.3 (*Count the keywords in Java source code*) Revise the program in Listing 21.7. If a keyword is in a comment or in a string, don't count it. Pass the Java file name from

CHAPTER

22

DEVELOPING EFFICIENT ALGORITHMS

Objectives

- To estimate algorithm efficiency using the Big O notation (§22.2).
- To explain growth rates and why constants and nondominating terms can be ignored in the estimation (§22.2).
- To determine the complexity of various types of algorithms (§22.3).
- To analyze the binary search algorithm (§22.4.1).
- To analyze the selection sort algorithm (§22.4.2).
- To analyze the Tower of Hanoi algorithm (§22.4.3).
- To describe common growth functions (constant, logarithmic, log-linear, quadratic, cubic, and exponential) (§22.4.4).
- To design efficient algorithms for finding Fibonacci numbers using dynamic programming (§22.5).
- To find the GCD using Euclid's algorithm (§22.6).
- To find prime numbers using the sieve of Eratosthenes (§22.7).
- To design efficient algorithms for finding the closest pair of points using the divide-and-conquer approach (§22.8).
- To solve the Eight Queens problem using the backtracking approach (§22.9).
- To design efficient algorithms for finding a convex hull for a set of points (§22.10).
- To design efficient algorithms for string matching using Boyer-Moore and KMP algorithms (§22.11).



```

56         else {
57             i++;
58         }
59     }
60
61     return fail;
62 }
63 }

```

The `match(text, pattern)` method (lines 17–40) tests whether `pattern` matches a substring in `text`. `i` indicates the current position in the text, which always moves forward. `k` indicates the current position in the pattern. If `text[i] == pattern[k]` (line 22), both `i` and `k` is incremented by 1 (lines 26–27). Otherwise, if `k > 0`, set `fail(k - 1)` to `k` so to slide the pattern to align `pattern[k]` with `text[i]` (line 31), else increase `i` by 1 (line 34).

The `getFailure(pattern)` method (lines 43–62) compares `pattern` with `pattern` to obtain, `fail[k]`, the length of the maximum prefix that is the suffix in `pattern[0..k]`. It initializes the array `fail` to zeros (line 44) and set `i` and `k` to 1 and 0, respectively (lines 45–46). `i` indicates the current position in the first pattern, which always moves forward. `k` indicates the current maximum length of a possible prefix that is also a suffix in `pattern[0..i]`. If `pattern[i] == pattern[k]`, set `fail[i]` to `k + 1` (line 49) and increase both `i` and `k` by 1 (lines 50–51). Otherwise, if `k > 0`, set `k` to `fail[k - 1]` to slide the second pattern to align `pattern[i]` in the first pattern with `pattern[k]` in the second pattern (line 54), else increase `i` by 1 (line 57).

To analyze the running time, consider three cases:

Case 1: `text[i]` is equal to `pattern[k]`. `i` is moved forward one position.

Case 2: `text[i]` is not equal to `pattern[k]` and `k` is 0. `i` is moved forward one position.

Case 3: `text[i]` is not equal to `pattern[k]` and `k > 0`. The pattern is moved at least one position forward.

In any case, either `i` is moved forward one position on the text or the pattern is shifted at least one position to the right. Therefore, the number of iterations in the while loop for the `match` method is at most $2n$. Similarly, the number of the iterations in the `getFailure` method is at most $2m$. Therefore the running time of the KMP algorithm is $O(n + m)$.

KEY TERMS

average-case analysis	840	dynamic programming approach	850
backtracking approach	864	exponential time	847
best-case input	840	growth rate	840
Big <i>O</i> notation	840	logarithmic time	846
brute force	851	quadratic time	843
constant time	841	space complexity	841
convex hull	867	time complexity	841
divide-and-conquer approach	862	worst-case input	840

CHAPTER SUMMARY

1. The *Big O notation* is a theoretical approach for analyzing the performance of an algorithm. It estimates how fast an algorithm's execution time increases as the input size increases, which enables you to compare two algorithms by examining their *growth rates*.
2. An input that results in the shortest execution time is called the *best-case* input, and one that results in the longest execution time is called the *worst-case* input. Best- and worst-case analyses are not representative, but worst-case analysis is very useful. You can be assured that the algorithm will never be slower than the worst case.

3. An *average-case analysis* attempts to determine the average amount of time among all possible input of the same size. Average-case analysis is ideal, but difficult to perform because for many problems, it is hard to determine the relative probabilities and distributions of various input instances.
4. If the time is not related to the input size, the algorithm is said to take *constant time* with the notation $O(1)$.
5. Linear search takes $O(n)$ time. An algorithm with the $O(n)$ time complexity is called a *linear algorithm* and it exhibits a linear growth rate. Binary search takes $O(\log n)$ time. An algorithm with the $O(\log n)$ time complexity is called a *logarithmic algorithm* and it exhibits a logarithmic growth rate.
6. The worst-time complexity for selection sort is $O(n^2)$. An algorithm with the $O(n^2)$ time complexity is called a *quadratic algorithm* and it exhibits a quadratic growth rate.
7. The time complexity for the Tower of Hanoi problem is $O(2^n)$. An algorithm with the $O(2^n)$ time complexity is called an *exponential algorithm*, and it exhibits an exponential growth rate.
8. A Fibonacci number at a given index can be found in $O(n)$ time using dynamic programming approach.
9. Dynamic programming is the process of solving subproblems, then combining the solutions of the subproblems to obtain an overall solution. The key idea behind dynamic programming is to solve each subproblem only once and store the results for subproblems for later use to avoid redundant computing of the subproblems.
10. Euclid's GCD algorithm takes $O(\log n)$ time.
11. All prime numbers less than or equal to n can be found in $O\left(\frac{n\sqrt{n}}{\log n}\right)$ time.
12. The closest pair can be found in $O(n \log n)$ time using the *divide-and-conquer approach*.
13. The divide-and-conquer approach divides the problem into subproblems, solves the subproblems, and then combines the solutions of the subproblems to obtain the solution for the entire problem. Unlike the dynamic programming approach, the subproblems in the divide-and-conquer approach don't overlap. A subproblem is like the original problem with a smaller size, so you can apply recursion to solve the problem.
14. The Eight Queens problem can be solved using backtracking.
15. The backtracking approach searches for a candidate solution incrementally, abandoning that option as soon as it determines the candidate cannot possibly be a valid solution, then looks for a new candidate.
16. A *convex hull* for a set of points can be found in $O(n^2)$ time using the gift-wrapping algorithm, and in $O(n \log n)$ time using the Graham's algorithm.
17. The brute force and Boyer-Moore string matching algorithms take $O(nm)$ time and the KMP string matching algorithm takes $O(n + m)$ time.

CHAPTER

23

SORTING

Objectives

- To study and analyze time complexity of various sorting algorithms (§§23.2–23.7).
- To design, implement, and analyze insertion sort (§23.2).
- To design, implement, and analyze bubble sort (§23.3).
- To design, implement, and analyze merge sort (§23.4).
- To design, implement, and analyze quick sort (§23.5).
- To design and implement a binary heap (§23.6).
- To design, implement, and analyze heap sort (§23.6).
- To design, implement, and analyze bucket and radix sorts (§23.7).
- To design, implement, and analyze external sort for files that have a large amount of data (§23.8).



CHAPTER SUMMARY

1. The worst-case complexity for a *selection sort*, *insertion sort*, *bubble sort*, and *quick sort* is $O(n^2)$.
2. The average- and worst-case complexity for a *merge sort* is $O(n \log n)$. The average time for a quick sort is also $O(n \log n)$.
3. *Heaps* are a useful data structure for designing efficient algorithms such as sorting. You learned how to define and implement a heap class, and how to insert and delete elements to/from a heap.
4. The time complexity for a *heap sort* is $O(n \log n)$.
5. *Bucket* and *radix sorts* are specialized sorting algorithms for integer keys. These algorithms sort keys using buckets rather than by comparing keys. They are more efficient than general sorting algorithms.
6. A variation of the merge sort—called an *external sort*—can be applied to sort large amounts of data from external files.

QUIZ

Answer the quiz for this chapter online at the book Companion Website.



PROGRAMMING EXERCISES

MyProgrammingLab™

Sections 23.3–23.5

- 23.1** (*Generic bubble sort*) Write the following two generic methods using bubble sort. The first method sorts the elements using the **Comparable** interface, and the second uses the **Comparator** interface.

```
public static <E extends Comparable<E>>
    void bubbleSort(E[] list)
public static <E> void bubbleSort(E[] list,
    Comparator<? super E> comparator)
```

- 23.2** (*Generic merge sort*) Write the following two generic methods using merge sort. The first method sorts the elements using the **Comparable** interface and the second uses the **Comparator** interface.

```
public static <E extends Comparable<E>>
    void mergeSort(E[] list)
public static <E> void mergeSort(E[] list,
    Comparator<? super E> comparator)
```

- 23.3** (*Generic quick sort*) Write the following two generic methods using quick sort. The first method sorts the elements using the **Comparable** interface, and the second uses the **Comparator** interface.

```
public static <E extends Comparable<E>>
    void quickSort(E[] list)
public static <E> void quickSort(E[] list,
    Comparator<? super E> comparator)
```

CHAPTER 25

BINARY SEARCH TREES

Objectives

- To design and implement a binary search tree (§25.2).
- To represent binary trees using linked data structures (§25.3).
- To search an element in a binary search tree (§25.4).
- To insert an element into a binary search tree (§25.5).
- To traverse elements in a binary tree (§25.6).
- To design and implement the **Tree** interface and the **BST** class (§25.7).
- To delete elements from a binary search tree (§25.8).
- To display a binary tree graphically (§25.9).
- To create iterators for traversing a binary tree (§25.10).
- To implement Huffman coding for compressing data using a binary tree (§25.11).



Tree class	root property. From the root, you can access all the nodes in the tree. The <code>Tree</code> class implements <code>Comparable</code>. The trees are comparable based on their weights. The compare order is purposely reversed (lines 93–100) so the smallest-weight tree is removed first from the heap of trees.
getHuffmanTree	The <code>getHuffmanTree</code> method returns a Huffman coding tree. Initially, the single-node trees are created and added to the heap (lines 50–54). In each iteration of the <code>while</code> loop (lines 56–60), two smallest-weight trees are removed from the heap and are combined to form a big tree, then the new tree is added to the heap. This process continues until the heap contains just one tree, which is our final Huffman tree for the text.
assignCode	The <code>assignCode</code> method assigns the code for each node in the tree (lines 34–45). The <code>getCode</code> method gets the code for each character in the leaf node (lines 26–31). The element <code>codes[i]</code> contains the code for character <code>(char) i</code>, where <code>i</code> is from 0 to 127. Note <code>codes[i]</code> is <code>null</code> if <code>(char) i</code> is not in the text.
getCode	



- 25.11.1** Every internal node in a Huffman tree has two children. Is it true?
- 25.11.2** What is a greedy algorithm? Give an example.
- 25.11.3** If the `Heap` class in line 50 in Listing 25.9 is replaced by `java.util.PriorityQueue`, will the program still work?
- 25.11.4** How do you replace lines 94–99 in Listing 25.11 using one line?

Key Terms

binary search tree	960	inorder traversal	963
binary tree	960	leaf	960
breadth-first traversal	964	length	960
depth	960	level	960
depth-first traversal	964	postorder traversal	964
greedy algorithm	986	preorder traversal	964
height	960	sibling	960
Huffman coding	985	tree traversal	963

Chapter Summary

1. A *binary search tree* (BST) is a hierarchical data structure. You learned how to define and implement a BST class, how to insert and delete elements into/from a BST, and how to traverse a BST using *inorder*, *postorder*, *preorder*, *depth-first*, and *breadth-first* searches.
2. An iterator is an object that provides a uniform way of traversing the elements in a container, such as a set, a list, or a *binary tree*. You learned how to define and implement iterator classes for traversing the elements in a binary tree.
3. *Huffman coding* is a scheme for compressing data by using fewer bits to encode characters that occur more frequently. The codes for characters are constructed based on the occurrence of characters in the text using a binary tree, called the *Huffman coding tree*.



Quiz

Answer the quiz for this chapter online at the book Companion Website.

CHAPTER

26

AVL TREES

Objectives

- To know what an AVL tree is (§26.1).
- To understand how to rebalance a tree using the LL rotation, LR rotation, RR rotation, and RL rotation (§26.2).
- To design the **AVLTree** class by extending the **BST** class (§26.3).
- To insert elements into an AVL tree (§26.4).
- To implement tree rebalancing (§26.5).
- To delete elements from an AVL tree (§26.6).
- To implement the **AVLTree** class (§26.7).
- To test the **AVLTree** class (§26.8).
- To analyze the complexity of search, insertion, and deletion operations in AVL trees (§26.9).



KEY TERMS

AVL tree	996	right-heavy	996
balance factor	996	RL rotation	997
left-heavy	996	rotation	996
LL rotation	996	RR rotation	996
LR rotation	997	well-balanced tree	996
perfectly balanced tree	996		

CHAPTER SUMMARY

1. An *AVL tree* is a *well-balanced* binary tree. In an AVL tree, the difference between the heights of two subtrees for every node is **0** or **1**.
2. The process for inserting or deleting an element in an AVL tree is the same as in a binary search tree. The difference is that you may have to rebalance the tree after an insertion or deletion operation.
3. Imbalances in the tree caused by insertions and deletions are rebalanced through subtree rotations at the node of the imbalance.
4. The process of rebalancing a node is called a *rotation*. There are four possible rotations: *LL rotation*, *LR rotation*, *RR rotation*, and *RL rotation*.
5. The height of an AVL tree is $O(\log n)$. Therefore, the time complexities for the **search**, **insert**, and **delete** methods are $O(\log n)$.



Quiz

Answer the quiz for this chapter online at the book Companion Website.

MyProgrammingLab™

PROGRAMMING EXERCISES

- *26.1** (*Display AVL tree graphically*) Write a program that displays an AVL tree along with its balance factor for each node.
- 26.2** (*Compare performance*) Write a test program that randomly generates 500,000 numbers and inserts them into a **BST**, reshuffles the 500,000 numbers and performs a search, and reshuffles the numbers again before deleting them from the tree. Write another test program that does the same thing for an **AVLTree**. Compare the execution times of these two programs.
- ***26.3** (*AVL tree animation*) Write a program that animates the AVL tree **insert**, **delete**, and **search** methods, as shown in Figure 26.2.
- **26.4** (*Parent reference for BST*) Suppose the **TreeNode** class defined in **BST** contains a reference to the node's parent, as shown in Programming Exercise 25.15. Implement the **AVLTree** class to support this change. Write a test program that adds numbers **1**, **2**, . . . , **100** to the tree and displays the paths for all leaf nodes.
- **26.5** (*The kth smallest element*) You can find the *k*th smallest element in a BST in $O(n)$ time from an inorder iterator. For an AVL tree, you can find it in $O(\log n)$ time. To achieve this, add a new data field named **size** in **AVLTreeNode** to store the number of nodes in the subtree rooted at this node. Note the size of a

CHAPTER 27

HASHING

Objectives

- To understand what hashing is and for what hashing is used (§27.2).
- To obtain the hash code for an object and design the hash function to map a key to an index (§27.3).
- To handle collisions using open addressing (§27.4).
- To know the differences among linear probing, quadratic probing, and double hashing (§27.4).
- To handle collisions using separate chaining (§27.5).
- To understand the load factor and the need for rehashing (§27.6).
- To implement **MyHashMap** using hashing (§27.7).
- To implement **MyHashSet** using hashing (§27.8).



load factor	1025	quadratic probing	1020
open addressing	1019	rehashing	1025
perfect hash function	1016	separate chaining	1023
polynomial hash code	1018		

CHAPTER SUMMARY

1. A *map* is a data structure that stores entries. Each entry contains two parts: a *key* and a *value*. The key is also called a *search key*, which is used to search for the corresponding value. You can implement a map to obtain $O(1)$ time complexity on searching, retrieval, insertion, and deletion using the hashing technique.
2. A *set* is a data structure that stores elements. You can use the hashing technique to implement a set to achieve $O(1)$ time complexity on searching, insertion, and deletion for a set.
3. *Hashing* is a technique that retrieves the value using the index obtained from a key without performing a search. A typical *hash function* first converts a search key to an integer value called a *hash code*, then compresses the hash code into an index to the *hash table*.
4. A *collision* occurs when two keys are mapped to the same index in a hash table. Generally, there are two ways for handling collisions: *open addressing* and *separate chaining*.
5. Open addressing is the process of finding an open location in the hash table in the event of collision. Open addressing has several variations: *linear probing*, *quadratic probing*, and *double hashing*.
6. The *separate chaining* scheme places all entries with the same hash index into the same location, rather than finding new locations. Each location in the separate chaining scheme is called a *bucket*. A bucket is a container that holds multiple entries.



QUIZ

Answer the quiz for this chapter online at the book Companion Website.

PROGRAMMING EXERCISES

- **27.1** (Implement *MyMap* using open addressing with linear probing) Create a new concrete class that implements *MyMap* using open addressing with linear probing. For simplicity, use $f(\text{key}) = \text{key} \% \text{size}$ as the hash function, where *size* is the hash-table size. Initially, the hash-table size is 4. The table size is doubled whenever the load factor exceeds the threshold (0.5). Test your new *MyHashMap* class using the code at https://liveexample.pearsoncmg.com/test/Exercise27_01.txt.
- **27.2** (Implement *MyMap* using open addressing with quadratic probing) Create a new concrete class that implements *MyMap* using open addressing with quadratic probing. For simplicity, use $f(\text{key}) = \text{key} \% \text{size}$ as the hash function, where *size* is the hash-table size. Initially, the hash-table size is 4. The table size is doubled whenever the load factor exceeds the threshold (0.5).
- **27.3** (Implement *MyMap* using open addressing with double hashing) Create a new concrete class that implements *MyMap* using open addressing with double hashing. For simplicity, use $f(\text{key}) = \text{key} \% \text{size}$ as the hash function, where *size*

CHAPTER

28

GRAPHS AND APPLICATIONS

Objectives

- To model real-world problems using graphs and explain the Seven Bridges of Königsberg problem (§28.1).
- To describe the graph terminologies: vertices, edges, simple graphs, weighted/unweighted graphs, and directed/undirected graphs (§28.2).
- To represent vertices and edges using lists, edge arrays, edge objects, adjacency matrices, and adjacency lists (§28.3).
- To model graphs using the **Graph** interface and the **UnweightedGraph** class (§28.4).
- To display graphs visually (§28.5).
- To represent the traversal of a graph using the **UnweightedGraph.SearchTree** class (§28.6).
- To design and implement depth-first search (§28.7).
- To solve the connected-circle problem using depth-first search (§28.8).
- To design and implement breadth-first search (§28.9).
- To solve the nine tails problem using breadth-first search (§28.10).



a graph model to get a BFS tree (line 11), obtains the shortest path from the initial node to the target node (lines 12–13), and displays the nodes in the path (lines 16–18).

- 28.10.1** How are the nodes created for the graph in `NineTailModel`?
- 28.10.2** How are the edges created for the graph in `NineTailModel`?
- 28.10.3** What is returned after invoking `getIndex("HTHTTTHHH".toCharArray())` in Listing 28.13? What is returned after invoking `getNode(46)` in Listing 28.13?
- 28.10.4** If lines 26 and 27 are swapped in Listing 28.13, `NineTailModel.java`, will the program work? Why not?



KEY TERMS

adjacency list	1053	parallel edge	1048
adjacency matrix	1052	Seven Bridges of Königsberg	1046
adjacent vertices	1048	simple graph	1048
breadth-first search	1067	spanning tree	1048
complete graph	1048	strongly connected graph	1048
cycle	1048	tree	1048
degree	1048	undirected graph	1047
depth-first search	1067	unweighted graph	1048
directed graph	1047	weakly connected graph	1048
graph	1046	weighted graph	1048
incident edges	1048		

CHAPTER SUMMARY

1. A *graph* is a useful mathematical structure that represents relationships among entities in the real world. You learned how to model graphs using classes and interfaces, how to represent vertices and edges using arrays and linked lists, and how to implement operations for graphs.
2. Graph traversal is the process of visiting each vertex in the graph exactly once. You learned two popular ways for traversing a graph: the *depth-first search* (DFS) and *breadth-first search* (BFS).
3. DFS and BFS can be used to solve many problems such as detecting whether a graph is connected, detecting whether there is a cycle in the graph, and finding the shortest path between two vertices.



QUIZ

Answer the quiz for this chapter online at the book Companion Website.

PROGRAMMING EXERCISES

MyProgrammingLab™

Sections 28.6–28.10

- *28.1** (*Test whether a graph is connected*) Write a program that reads a graph from a file and determines whether the graph is connected. The first line in the file contains a number that indicates the number of vertices (**n**). The vertices are labeled as **0, 1, . . . , n-1**. Each subsequent line, with the format **u v1 v2 . . .**, describes edges (**u, v1**), (**u, v2**), and so on. Figure 28.21 gives the examples of two files for their corresponding graphs.

CHAPTER

29

WEIGHTED GRAPHS AND APPLICATIONS

Objectives

- To represent weighted edges using adjacency matrices and adjacency lists (§29.2).
- To model weighted graphs using the **WeightedGraph** class that extends the **UnweightedGraph** class (§29.3).
- To design and implement the algorithm for finding a minimum spanning tree (§29.4).
- To define the **MST** class that extends the **SearchTree** class (§29.4).
- To design and implement the algorithm for finding single-source shortest paths (§29.5).
- To define the **ShortestPathTree** class that extends the **SearchTree** class (§29.5).
- To solve the weighted nine tails problem using the shortest-path algorithm (§29.6).



CHAPTER SUMMARY

1. You can use adjacency matrices or lists to store weighted edges in graphs.
2. A spanning tree of a graph is a subgraph that is a tree and connects all vertices in the graph.
3. Prim's algorithm for finding a minimum spanning tree works as follows: the algorithm starts with a spanning tree **T** that contains an arbitrary vertex. The algorithm expands the tree by adding a vertex with the minimum-weight edge incident to a vertex already in the tree.
4. Dijkstra's algorithm starts the search from the source vertex and keeps finding vertices that have the shortest path to the source until all vertices are found.



Quiz

Answer the quiz for this chapter online at the book Companion Website.

MyProgrammingLab™

PROGRAMMING EXERCISES

- *29.1** (*Kruskal's algorithm*) The text introduced Prim's algorithm for finding a minimum spanning tree. Kruskal's algorithm is another well-known algorithm for finding a minimum spanning tree. The algorithm repeatedly finds a minimum-weight edge and adds it to the tree if it does not cause a cycle. The process ends when all vertices are in the tree. Design and implement an algorithm for finding an MST using Kruskal's algorithm.
- *29.2** (*Implement Prim's algorithm using an adjacency matrix*) The text implements Prim's algorithm using lists for adjacent edges. Implement the algorithm using an adjacency matrix for weighted graphs.
- *29.3** (*Implement Dijkstra's algorithm using an adjacency matrix*) The text implements Dijkstra's algorithm using lists for adjacent edges. Implement the algorithm using an adjacency matrix for weighted graphs.
- *29.4** (*Modify weight in the nine tails problem*) In the text, we assign the number of the flips as the weight for each move. Assuming the weight is three times of the number of flips, revise the program.
- *29.5** (*Prove or disprove*) The conjecture is that both **NineTailModel** and **WeightedNineTailModel** result in the same shortest path. Write a program to prove or disprove it. (*Hint*: Let **tree1** and **tree2** denote the trees rooted at node **511** obtained from **NineTailModel** and **WeightedNineTailModel**, respectively. If the depth of a node **u** is the same in **tree1** and in **tree2**, the length of the path from **u** to the target is the same.)
- **29.6** (*Weighted 4 × 4 16 tails model*) The weighted nine tails problem in the text uses a 3 × 3 matrix. Assume that you have 16 coins placed in a 4 × 4 matrix. Create a new model class named **WeightedTailModel16**. Create an instance of the model and save the object into a file named **WeightedTailModel16.dat**.
- **29.7** (*Weighted 4 × 4 16 tails*) Revise Listing 29.9, **WeightedNineTail.java**, for the weighted 4 × 4 16 tails problem. Your program should read the model object created from the preceding exercise.